ATmega2560 Project Cards

Functionality and technical implementation

Numbering follows the repository. Projects 20 and 23 are hardware exercises without separate directories.

01 Blinking the Onboard LED

What the project does

The onboard LED stays on for 0.5 seconds and off for 0.5 seconds, repeating continuously.

Technical implementation and concepts

DDRB configures PB7 as an output, and the main loop changes its PORTB level with 500 ms delays using _delay_ms. This introduces register-level GPIO, bit operations, and building and running a minimal AVR C program.

Code: code/01_bare_metal_blink

View Project 01 code

02 Alternating Onboard and External LEDs

What the project does

The onboard LED turns on for 0.2 seconds and off for 0.2 seconds. An external yellow LED then follows the same pattern. The two LEDs repeat this sequence continuously.

Technical implementation and concepts

PB7 and PB4 are configured as outputs. Separate bit masks control the onboard and external LEDs, while 200 ms delays sequence their operation, demonstrating independent control of multiple pins on one port.

Code: code/02_external_led

View Project 02 code

03 Setting LED Blink Counts and Intervals

What the project does

The LED blinks quickly three times, stays off for one second, then blinks slowly twice and remains off. Changing the blink counts and intervals produces different light sequences.

Technical implementation and concepts

The wait_ms and blink functions encapsulate delay and output logic. Count and interval parameters, const values, and fixed-width integer types organize the fast-blink, pause, and slow-blink sequence.

Code: code/03_blink_function

View Project 03 code

04 Cycling Through Three LED Colors

What the project does

Red, green, and blue LEDs light up in sequence, with a brief interval when all three are off between colors. The sequence repeats continuously. Changing the settings for each step changes the order and timing of the display.

Technical implementation and concepts

State arrays define the three LED outputs at each step, while a duration array defines 50 ms light intervals and 20 ms all-off intervals. A for loop indexes the arrays, separating output states from timing in a table-driven program.

Code: code/04_traffic_light

View Project 04 code

05 Button-Controlled LED with a Reliable Input

What the project does

The LED lights while the button is held and turns off when it is released. The program continuously distinguishes pressing from releasing, with a clear released state even when the button is untouched.

Technical implementation and concepts

PB4 is configured as an input with an internal pull-up, making a press active-low. Reading PINB controls the PB5 output, combining input-direction configuration, pull-ups, and logical inversion to map button level to LED output.

Code: code/05_button_hold_led

View Project 05 code

06 Toggling an LED on Each Button Press

What the project does

One press turns the LED on; another turns it off. Holding the button does not repeatedly switch the light. This version does not yet filter the brief signal changes caused by mechanical contact bounce.

Technical implementation and concepts

The program retains the previous button reading, compares it with the current reading, and toggles the LED only on a falling edge. This distinguishes a sustained logic level from a discrete event; mechanical debouncing is not yet implemented.

Code: code/06_button_toggle_led

View Project 06 code

07 Selecting Four LED Behaviors

What the project does

Successive presses select off, steady on, fast blinking, and slow blinking. Each blinking mode performs a short sequence and then leaves the LED off. The next press selects the following behavior.

Technical implementation and concepts

A mode variable increments on each falling edge and wraps to zero. A switch/case structure selects off, steady on, or blink sequences with different delays, implementing state-driven output control.

Code: code/07_button_statemachine

View Project 07 code

08 Displaying Button Bounce as LED Flashes

What the project does

After a button press, the program records how many times the button signal changes within a short interval. Once the button is released, the LED flashes that many times, showing the signal changes detected during a single press.

Technical implementation and concepts

A press starts a window of 100 samples spaced 50 microseconds apart. The program counts level transitions and displays the count through LED flashes after release, introducing mechanical-bounce sampling and event counting.

Code: code/08_measure_button_bounce

View Project 08 code

09 Reducing Duplicate Button Triggers

What the project does

The program waits for the button input to settle before accepting a change. The first accepted press lights the LED, and the next turns it off, reducing cases where one physical press is counted as several actions.

Technical implementation and concepts

Separate variables retain raw readings, candidate state, and confirmed state. A stability counter must reach its threshold before a transition is committed; confirmed events then toggle the LED, establishing incremental debouncing and edge-counting logic.

Code: code/09_nonblocking_debounce_fsm

View Project 09 code

10 Different Light Responses to Short and Long Presses

What the project does

A short press followed by release makes the LED flash quickly three times. Holding the button longer before releasing it produces five slow flashes. One button therefore selects two different light responses.

Technical implementation and concepts

The program accumulates press duration after debouncing and selects a fast- or slow-blink function on release using a threshold. Button lifecycle handling, timing decisions, and output actions are kept separate.

Code: code/10_short_long_press

View Project 10 code

11 Controlling Two LEDs with Two Buttons

What the project does

Each button controls its own LED. The two lights retain their individual states, so the buttons can be operated in turn and the LEDs can remain in different on/off combinations.

Technical implementation and concepts

Each input has independent debounce counters, confirmed states, and press counts. These control PB6 and PB7 separately, extending single-input state handling without interference from shared variables.

Code: code/11_dual_button_dual_led

View Project 11 code

12 Viewing Button Events and Counts on a Computer

What the project does

The computer displays a startup message when the board begins running. Operating either button displays its event and total press count, while both LEDs remain controllable.

Technical implementation and concepts

USART0 is configured for transmission at the selected baud rate. Startup messages, button events, and counts are encoded as ASCII, combining GPIO control with serial debugging and integer-to-decimal text conversion.

Code: code/12_uart_tx_debug

View Project 12 code

13 Controlling Two LEDs with Computer Commands

What the project does

Typing on_1, off_1, on_2, or off_2 switches the two LEDs independently. The status command reports their current states, and help lists the commands. The program also handles backspace and reports commands that are too long or unrecognized.

Technical implementation and concepts

A bounded character buffer receives UART commands and handles backspace, line endings, and excess input. String comparisons dispatch LED control and status requests, forming a basic command parser.

Code: code/13_uart_rx_command

View Project 13 code

14 Switching a Buzzer with Computer Commands

What the project does

Sending on from the computer makes the buzzer sound; sending off stops it. Entered text is echoed back, and an error message appears if the command is too long.

Technical implementation and concepts

UART line reception and length checking are reused to recognize on/off commands, set the buzzer pin level, and echo the input. This maps text commands to an active-buzzer output.

Code: code/14_buzzer_square_wave

View Project 14 code

15 Detecting Button Changes While an LED Blinks

What the project does

The LED changes between on and off about once per second. Pressing or releasing the button during blinking still produces a change message on the computer, without waiting for a complete light sequence to finish.

Technical implementation and concepts

The main loop alternates timing updates, periodic LED checks, and button debouncing. Elapsed-time comparisons update outputs and emit state events instead of blocking for an entire blink sequence, creating a cooperative task structure.

Code: code/15_nonblocking_multitask

View Project 15 code

16 Timing an LED with a Hardware Timer

What the project does

The LED repeats a pattern of roughly one second on and one second off. Switching follows the time continuously recorded by the development board.

Technical implementation and concepts

Timer1 operates in CTC mode, with its prescaler and OCR1A configured for a 1 ms compare-match interrupt. The ISR updates system_ms, and the main loop atomically reads a time snapshot to schedule the LED.

Code: code/16_timer_1ms_tick

View Project 16 code

17 Continuously Reporting System Uptime

What the project does

The LED changes state once per second while the computer repeatedly displays SYSTEM ALIVE and the elapsed time in milliseconds. The increasing time helps show whether the program is still running and whether its numeric output is correct.

Technical implementation and concepts

An ISR updates a volatile uint32_t millisecond counter. ATOMIC_BLOCK provides a consistent snapshot for decimal-text output, addressing interrupt-shared data, multibyte atomicity, and numeric type width.

Code: code/17-isr-volatile-atomicity

View Project 17 code

18 Running LED, Button, and Reporting Tasks Together

What the project does

The LED changes state every 0.5 seconds, while button presses accumulate in a counter. About every two seconds, the computer reports the current time, LED state, and button count. These functions continue while the button is being used.

Technical implementation and concepts

A shared 1 ms tick supports separate timestamps for the LED, serial reports, and button debouncing. Tasks run cooperatively at different periods, using uint32_t counters and serial status output to observe execution.

Code: code/18_fixed_tick_tack_scheduler

View Project 18 code

19 Generating Stepper Pulses and Direction Changes

What the project does

The board continuously sends motor-control signals and changes the direction setting about every five seconds. The computer receives a running message about every four seconds. The signals can also be measured without connecting a motor.

Technical implementation and concepts

Timer1 output compare generates STEP pulses, while Timer3 maintains the system timebase. The main loop schedules DIR changes and reports independently, allowing hardware pulse generation and periodic software tasks to proceed together.

Code: code/19_step_dir_pulse_generator

View Project 19 code

20 First Stepper Motor Operation

What the project does

A stepper motor is connected through a driver and a separate power supply so it can turn in response to the board's signals. Changing the direction signal allows its rotation to be observed in the opposite direction. This hardware exercise reuses Project 19.

Technical implementation and concepts

The original exercise connects STEP/DIR signals to an external current-limiting driver and checks phase current, separate power, and common ground. It connects logic-level control to a physical stepper drive using the Project 19 workspace.

Hardware exercise: reuses Project 19; the repository has no separate Project 20 directory.

21 Receiving Commands During Motor Operation

What the project does

Computer commands can start or stop the motor, adjust its speed, reverse direction, and request its status. New commands are still received while the motor is running. Periodic reports show speed, direction, and whether the motor is running.

Technical implementation and concepts

Hardware maintains STEP output while the main loop incrementally receives commands in a persistent character buffer. Motor interfaces handle start/stop, speed, and direction, with periodic status reports, so command reception does not block motion control.

Code: code/21_nonblocking_stepper_control

View Project 21 code

22 Stepper Acceleration, Deceleration, and an Automatic Demo

What the project does

The motor gradually accelerates to its selected speed and slows down when stopping. The computer can change speed and acceleration and display phase changes and speed logs. An automatic demo accelerates, runs for a set interval, and decelerates to a stop.

Technical implementation and concepts

A motion state machine updates the step-frequency ramp over time and distinguishes acceleration, cruise, and deceleration. A demo task uses state transitions and timing to request a stop, while logs record speed and phase changes.

Code: code/22_stepper_acceleration_ramp

View Project 22 code

23 Comparing Motor Smoothness, Load, and Heating

What the project does

This experiment compares motor smoothness, sound, and heating under different driver settings and loads. The experiment log should record speed, load, temperature, and any missed steps so the settings can be compared.

Technical implementation and concepts

The original exercise varies microstepping, current, and load and records speed, temperature rise, and missed steps to compare smoothness and load capacity. The repository does not contain a separate experiment project.

Hardware exercise from the original project plan; the repository has no separate Project 23 directory.

24 Stepper Homing, Limits, and Emergency Stop

What the project does

The motor can find its starting position, back away after reaching a switch, and approach again slowly to establish a reference. Reaching a limit switch or pressing the emergency stop stops motion and produces a status report. Switch problems or failure to find the starting reference also produce fault reports.

Technical implementation and concepts

Limit checking, ESTOP, and homing are separate modules. Interrupt-level guard checks stop pulses, while a homing state machine handles search, backoff, slow approach, and timeout, establishing fault states and exception paths.

Code: code/24_stepper_limits_homing_estop

View Project 24 code

25 Positioning and Repeated Round Trips

What the project does

Once the motor has found its starting reference, a computer command can send it to a specified position. Two points and a repeat count can also define a round-trip task. Motion status, configuration, and logs remain available while stop, limit, and emergency-stop inputs are handled. Long help messages are sent in portions so reporting does not hold up other work.

Technical implementation and concepts

Positioning, repeat, homing, and safety tasks organize motion. The reporting module retains pending requests and sends output according to free TX-buffer space, preventing long messages from blocking the main loop and deceleration updates.

Code: code/25_stepper_safe_system_integration

View Project 25 code

26 Receiving Ammonia Sensor Data and Reporting Its Status

What the project does

The computer shows incoming raw data, concentration readings, and states such as warming up, valid, or out of date. Invalid or incomplete data is counted as an error. Older readings are identified as such, while sensor reception and motion tasks continue separately.

Technical implementation and concepts

A dedicated UART validates frame headers, length, checksum, and fields before committing a complete concentration value. Reception and protocol adaptation are separated, with sample-age and warm-up tracking. ZE03 is the current default; ZE801 support awaits protocol confirmation.

Code: code/26_ze801_nh3_uart_sensor

View Project 26 code

27 Fixed Brightness and a Fading LED

What the project does

The LED can be switched off, held at a selected brightness, or made to fade repeatedly up and down. The computer can adjust the fading cycle and report the current brightness and mode. Motor and serial tasks continue during the light effect.

Technical implementation and concepts

Timer4 hardware PWM maintains the output, while an LED task updates duty cycle from system time. Fixed brightness and an adjustable fading period separate carrier generation from brightness-envelope updates.

Code: code/27_nonblocking_pwm_breathing_led

View Project 27 code

28 Knob-Controlled Brightness and Voltage Readout

What the project does

In knob-control mode, turning the knob changes LED brightness. The computer can show the raw reading, calculated voltage, update time, and corresponding brightness so the changes can be observed together.

Technical implementation and concepts

Periodic potentiometer ADC samples are converted to millivolts using the reference voltage and mapped to PWM duty cycle. Timestamps and validity flags separate acquisition, data state, and dimming output.

Code: code/28_potentiometer_led_brightness

View Project 28 code

29 Reading Two Inputs and Selecting the Dimming Source

What the project does

The computer shows readings, voltages, and update times for two inputs and lets the user choose which one controls LED brightness. After switching sources, the LED follows the newly selected input while readings from the other channel are retained.

Technical implementation and concepts

A shared ADC scans channels through the MUX and stores independent sample snapshots. The dimming module selects its source and checks channel validity, extending acquisition to multiple inputs.

Code: code/29_adc_register_dual_channel

View Project 29 code

30 Smoothing and Calibrating Knob Readings

What the project does

The computer can compare raw readings, averages, calibrated values, and the resulting brightness. Smoothing strength, the threshold for brightness changes, and the knob's usable endpoints can be adjusted to observe how much flicker is reduced and whether the response becomes slower.

Technical implementation and concepts

Each input maintains a moving-average window, calibration endpoints, and a hysteresis threshold. Calibration and mapping update a held brightness value, allowing filter stability to be compared with step-response delay.

Code: code/30_adc_filter_hysteresis_calibration

View Project 30 code

31 Automatic LED Control from Ambient Light

What the project does

In automatic mode, the light turns on as the surroundings become dark and turns off as they brighten. Different turn-on and turn-off conditions reduce repeated switching near the boundary. The computer also shows light readings, settings, and the number of switches.

Technical implementation and concepts

Averaged and normalized light-sensor readings drive hysteresis control with separate on/off thresholds. Polarity and data freshness are checked before applying the input to a nonblocking automatic-lighting task.

Code: code/31_light_sensor_auto_led

View Project 31 code

32 Playing Tones, a Scale, and a Melody

What the project does

Computer commands can play a tone at a selected pitch and duration, a complete scale, or a preset short melody, and can stop playback early. Other tasks remain active during playback. Status reports show the current note, frequency, and remaining time.

Technical implementation and concepts

A timer generates the requested square-wave frequency, while a PROGMEM note table stores frequencies and durations. A playback state machine schedules notes, rests, and gaps independently of waveform generation.

Code: code/32_nonblocking_buzzer_notes

View Project 32 code

33 Counting External Events and Comparing Detection Methods

What the project does

The computer displays event counts from two detection methods, their rates per second, and the difference between the counts. Button mode separately counts accepted presses. Faster input signals can reveal missed events. Counting can be started, stopped, or cleared.

Technical implementation and concepts

INT5 and main-loop polling count events on the same input, with periodic rate and count-difference calculations. Button mode adds debouncing, and atomic snapshots support comparison of interrupt edge capture with polling.

Code: code/33_external_interrupt_event_counter

View Project 33 code

34 Ultrasonic Distance Measurement and Proximity Alerts

What the project does

The computer shows distance, the latest measurement time, and measurement status. Raw, mean-filtered, and median-filtered distances can be compared. A message appears when an object enters the selected proximity range. Missing echoes, timeouts, and abnormal signals produce distinct status reports.

Technical implementation and concepts

A state machine triggers measurements, waits for rising and falling echo edges, measures pulse width, and converts it to distance. Timeouts, mean/median filters, and hysteretic alerts distinguish measurement errors from valid distance changes.

Code: code/34_nonblocking_ultrasonic_distance

View Project 34 code

35 Scanning Connected Device Addresses

What the project does

The computer lists devices that respond on the shared connection, identified by their addresses. It also shows scan progress, device count, and elapsed time. A scan can be canceled. Statistics and error messages identify missing responses, timeouts, and connection faults.

Technical implementation and concepts

An I2C state machine probes 7-bit addresses and records ACK/NACK responses. START, address, and STOP transactions proceed incrementally, with timeout, bus-error, cancellation, and retry tracking.

Code: code/35_nonblocking_i2c_scanner

View Project 35 code

36 MPU-6050 Identification and Three-Axis Acceleration

What the project does

The computer shows the sensor's identity and acceleration readings along the X, Y, and Z axes. Single readings or continuous sampling are available, with an indication of whether data is still fresh. Failed reads retain the previous complete sample and its age while recovery is attempted. Repeated failures eventually stop retries and produce a report.

Technical implementation and concepts

MPU-6050 identity checks, reset, configuration readback, and three-axis sampling run in sequence, committing only complete results. Recovery uses bounded retries and backoff; only a valid sample clears the consecutive-failure counter.

Code: code/36_nonblocking_i2c_sensor_registers

View Project 36 code

37 SPI Data Verification and Voltage Output

What the project does

The computer can start multiple send/receive tests, compare transmitted data with returned data, and show passes, mismatches, or errors. With an MCP4921 voltage-output module connected, it can send a target value and report the last value sent and its theoretical voltage. Tests and voltage commands share the communication interface in a coordinated order.

Technical implementation and concepts

The SPI loopback task sends test patterns in chunks and checks them byte by byte. A DAC task sends MCP4921 control words, with bus ownership coordinating access. Loopback verification, completed transmission, and measured analog output remain distinct.

Code: code/37_nonblocking_spi_loopback_dac

View Project 37 code

38 A Multi-Page OLED Status Display

What the project does

The small display switches among motion, sensor, and system pages. These show motor position and speed, distance and acceleration readings, counts, and communication states. Emergency stops, limits, stale data, and device failures produce notices. The display updates changed content, and communication failures produce retry messages on the computer.

Technical implementation and concepts

The SSD1306 interface renders state snapshots into a framebuffer row by row. Dirty-region tracking and incremental I2C transfers support partial refreshes, while bounded retries with backoff handle faults and limit display workload.

Code: code/38_nonblocking_oled_dashboard

View Project 38 code

39 Servo Angle, Speed, and Knob Tracking

What the project does

Computer commands set a servo's target angle and turning speed, or make its target follow a selected knob input. The commanded angle changes gradually. An emergency stop or invalid knob data stops control pulses and reports the reason; a new command is needed to resume.

Technical implementation and concepts

Angles map to servo pulse widths, with rate-limited command updates at fixed time steps. ESTOP and ADC data age are checked before output; faults stop pulses. Target angle, commanded output, and actual position feedback are explicitly distinguished.

Code: code/39_nonblocking_servo_control

View Project 39 code

40 Automatic Round Trips and Integrated System Control

What the project does

After the motor finds its starting reference, the user sets positions A and B and a cycle count from the computer. The motor first moves to A, then completes the requested A-to-B-to-A cycles and finishes at A. Reports show the current phase, completed cycles, target, and position tracked by step count, and record how the run ended. Completion, manual stops, and faults have distinct outcomes. The system coordinates manual commands with automatic movement and prevents changes to the plan during a run. Sensors, the display, and other modules continue operating while the system summarizes their states.

Technical implementation and concepts

A system state machine coordinates homing, plan setup, positioning, and automatic round trips. Task arbitration constrains manual/automatic operations, and runlog records progress and termination reasons. Peripheral tasks advance independently, with system-level state aggregation.

Code: code/40_mega_integrated_control_system

View Project 40 code